HMI_FINAL.py — Dokumentasi HMI Auto Dispensing Iodium

1. Judul & Deskripsi

`HMI_FINAL.py` adalah aplikasi desktop berbasis **Python 3 + Tkinter/ttk** yang berfungsi sebagai antarmuka operator (HMI) untuk sistem *auto dispensing* kapsul iodium. Aplikasi menerima input volume iodium per kapsul, faktor kalibrasi, dan jumlah kapsul, lalu mengirim perintah berbasis **STEP** ke Arduino melalui komunikasi serial.

Perintah utama yang dikirim HMI adalah:

```
<INIT:stepsPerCapsule;jumlahCapsule>
```

HMI tidak menggerakkan motor secara langsung. Seluruh eksekusi mekanis, homing, persiapan iodium, pengisian kapsul, dan penutupan dilakukan oleh firmware Arduino. HMI bertanggung jawab atas validasi input, konversi volume ke step, pengiriman perintah, pemantauan status, dan pencatatan log.

> **Peringatan keselamatan:** dokumen ini menjelaskan perilaku kode, bukan validasi dosis, proses farmasi, atau kelayakan klinis/radiologis. Pastikan mekanik, limit switch, *emergency stop*, prosedur operasi, dan batas volume telah divalidasi oleh personel berwenang. Firmware Arduino yang kompatibel menjalankan gerakan secara *blocking*, sehingga perintah pembatalan tidak dapat diproses selama siklus otomatis berlangsung.

- **Berkas:** `HMI_FINAL.py`
- **Jumlah baris:** 496
- **Bahasa:** Python 3
- **GUI:** Tkinter dan ttk
- **Komunikasi:** pyserial, 9600 baud
- **Log:** `logs/dispensing_log.csv`

2. Ringkasan Fungsi Program

Fungsi utama aplikasi:

1. Menyediakan form input volume per kapsul, faktor kalibrasi, dan jumlah kapsul.
2. Memvalidasi input: volume > 0, faktor > 0, jumlah kapsul 1–5.
3. Membatasi volume maksimum **10,0 µL** pada alur INIT.
4. Mengonversi volume ke step dengan rumus `step = round(volume × faktor)`.
5. Mengirim perintah `<INIT:step;jumlah>` ke Arduino.
6. Menampilkan seluruh trafik serial TX/RX pada panel *Serial Monitor*.
7. Memantau status proses melalui kata kunci pada data serial yang diterima.
8. Menyediakan progress bar dan status pengiriman.
9. Melakukan **auto-reconnect** ke port Arduino setiap 2 detik.
10. Menyediakan tombol **Refresh COM** untuk koneksi manual.

11. Menyediakan pengiriman perintah manual bebas.
12. Menyediakan toggle tema gelap/terang.
13. Mencatat aktivitas ke file CSV.
14. Menutup koneksi serial dengan aman saat aplikasi keluar.

Fitur yang **tidak** disediakan saat ini: pemilihan port/baud dari UI, tombol *emergency stop*, pembatalan siklus, penyimpanan pengaturan, rotasi log, dan pengujian otomatis.

3. Prasyarat & Dependensi

Kebutuhan	Keterangan
Python 3.x	Termasuk modul standar `tkinter`, `threading`, `csv`, `datetime`, `os`.
`pyserial`	Diperlukan untuk `import serial` dan `serial.tools.list_ports`.
Arduino + firmware	Firmware harus menerima perintah `<INIT:step;jumlah>`; contoh yang sesuai adalah `I_131_SingleFillSingleDispense_v6_working`.
Kabel USB/serial	Port harus dikenali sistem operasi.
Driver USB-serial	Misalnya driver CH340/FT232 sesuai board.

Instalasi dependensi:

```
py -m pip install pyserial
```

`tkinter` biasanya sudah tersedia pada instalasi Python Windows. Tidak ada library GUI eksternal, file konfigurasi, atau aset gambar yang dibutuhkan.

Menjalankan aplikasi:

```
py HMI_FINAL.py
```

Jalankan dari direktori proyek agar folder `logs` dibuat di lokasi yang diharapkan. Jika aplikasi dijalankan dari direktori lain, log akan mengikuti direktori kerja tersebut karena `LOG_DIR` bersifat relatif.

4. Arsitektur & Komponen

4.1 Komponen utama

Komponen	Implementasi	Tanggung jawab
Antarmuka pengguna	Tkinter/ttk	Input, tombol, status, monitor, progress bar, tema.

Komponen	Implementasi	Tanggung jawab
Koneksi serial	`serial.Serial`	Komunikasi 9600 baud dengan Arduino.
Serial reader thread	`serial_reader_thread()`	Membaca data masuk dan memperbarui monitor/status.
Timeout watcher	`watch_timeout()` di dalam `send_init_command()`	Menghentikan progress bar setelah 6 detik atau saat status DONE/ERROR.
Auto-reconnect	`auto_reconnect_loop()` + `root.after()`	Mendeteksi dan membuka port secara berkala.
Logger	`log_to_csv()`	Menulis riwayat ke `logs/dispensing_log.csv`.
Lock serial	`serial_lock`	Mencegah akses baca/tulis serial secara bersamaan.

4.2 Variabel global

Variabel	Nilai awal	Fungsi
`ser`	`None`	Objek `serial.Serial` aktif.
`serial_lock`	`threading.Lock()`	Sinkronisasi akses ke `ser`.
`running`	`True`	Penanda loop reader masih berjalan.
`theme_dark`	`True`	Status tema saat ini.

4.3 Peta widget UI

Widget	Lokasi kode	Fungsi
`root`	`HMI_FINAL.py:350`	Jendela utama 720×640, minimum 720×640.
`title`	`HMI_FINAL.py:359`	Judul "AUTO DISPENSING IODIUM".
`entry_volume`	`HMI_FINAL.py:380`	Input volume per kapsul (µL).
`kal_var`, `entry_kal`	`HMI_FINAL.py:373-375`	Faktor kalibrasi, default `2033` step/µL.
`jumlah_var`, `jumlah_combo`	`HMI_FINAL.py:386-389`	Pilihan jumlah kapsul 1–5, default 1.
`btn_send`	`HMI_FINAL.py:395`	Tombol "KIRIM KE ARDUINO".
`progress`	`HMI_FINAL.py:398`	Progress bar indeterminate, panjang 220 px.
`status_label`	`HMI_FINAL.py:402`	Label status proses yang terlihat operator.
`status_var`	`HMI_FINAL.py:404`	Variabel status internal untuk watcher/timeout.
`monitor`	`HMI_FINAL.py:410`	Text widget serial monitor, read-only.
`entry_command`	`HMI_FINAL.py:420`	Input perintah manual.

Widget	Lokasi kode	Fungsi
`btn_send_cmd`	`HMI_FINAL.py:422`	Tombol kirim perintah manual.
`theme_btn`	`HMI_FINAL.py:426`	Tombol toggle tema.
`btn_refresh`	`HMI_FINAL.py:456`	Tombol "Refresh COM".
`status_bar`	`HMI_FINAL.py:460`	Label status di bagian bawah jendela.

Catatan: `status_bar` dibuat dengan teks awal `"Ready."`, tetapi tidak pernah diperbarui oleh kode setelahnya. Status yang berubah selama operasi adalah `status_label` dan `status_var`.

4.4 Model thread

Thread	Dibuat di	Sifat	Pekerjaan
Serial reader	`HMI_FINAL.py:469-470`	Daemon	Loop membaca serial selama `running` bernilai `True`.
Timeout watcher	`HMI_FINAL.py:201`	Daemon	Memantau status dan menghentikan progress setelah 6 detik.
Auto-reconnect	`HMI_FINAL.py:472`	Callback `root.after`	Berjalan di thread utama setiap 2 detik.

5. Konfigurasi

Konstanta	Nilai	Lokasi	Keterangan
`BAUDRATE`	`9600`	`HMI_FINAL.py:18`	Baud rate komunikasi serial.
`RECONNECT_INTERVAL_MS`	`2000`	`HMI_FINAL.py:19`	Interval pengecekan auto-reconnect (ms).
`SERIAL_READ_SLEEP`	`0.1`	`HMI_FINAL.py:20`	Jeda saat tidak ada data serial (detik).
`MAX_VOLUME_UL`	`10.0`	`HMI_FINAL.py:21`	Batas keras volume per kapsul pada alur INIT (µL).
`LOG_DIR`	`"logs"`	`HMI_FINAL.py:22`	Direktori log relatif terhadap direktori kerja.
`LOG_FILE`	`logs/dispensing_log.csv`	`HMI_FINAL.py:23`	Path file log.
Faktor default	`2033`	`HMI_FINAL.py:373`	Nilai awal input faktor kalibrasi.
Pilihan kapsul	`1`–`5`	`HMI_FINAL.py:387`	Nilai yang tersedia pada Combobox.

Contoh: volume `10 µL` dengan faktor `2033` menghasilkan `20330` step per kapsul, sesuai contoh perintah pada dokumentasi firmware `<INIT:20330;4>`.

6. Referensi Fungsi

Fungsi	Lokasi	Tanggung jawab
`log_to_csv()`	`HMI_FINAL.py:36`	Membuat folder/log, menulis header bila baru, menambahkan baris log.
`detect_arduino_port()`	`HMI_FINAL.py:67`	Memindai port serial dan mengembalikan port yang cocok atau `None`.
`safe_write()`	`HMI_FINAL.py:81`	Menulis string ke serial dengan lock; mencatat `TX → ...`.
`append_monitor()`	`HMI_FINAL.py:95`	Menambahkan teks ke serial monitor dan melakukan auto-scroll.
`try_parse_float()`	`HMI_FINAL.py:102`	Mengubah string menjadi float; mengembalikan `None` bila gagal.
`close_serial()`	`HMI_FINAL.py:109`	Menutup `ser` dan mengesetnya menjadi `None`.
`send_init_command()`	`HMI_FINAL.py:123`	Validasi, konversi µL→step, pengiriman INIT, progress, dan timeout.
`send_manual_command()`	`HMI_FINAL.py:209`	Mengirim isi kolom perintah manual apa adanya.
`serial_reader_thread()`	`HMI_FINAL.py:227`	Loop pembacaan serial dan pemanggilan handler status.
`handle_incoming_status()`	`HMI_FINAL.py:251`	Memetakan kata kunci serial ke status UI dan menulis log.
`auto_reconnect_loop()`	`HMI_FINAL.py:292`	Mencoba membuka port secara berkala melalui `root.after`.
`apply_theme_dark()`	`HMI_FINAL.py:321`	Menerapkan warna tema gelap.
`apply_theme_light()`	`HMI_FINAL.py:329`	Menerapkan warna tema terang.
`toggle_theme()`	`HMI_FINAL.py:337`	Berganti tema dan mengubah teks tombol.
`manual_refresh()`	`HMI_FINAL.py:430`	Memindai dan membuka port saat tombol Refresh COM ditekan.
`on_closing()`	`HMI_FINAL.py:477`	Konfirmasi keluar, menghentikan reader, menutup serial, menghancurkan UI.

6.1 Detail fungsi penting

```
log_to_csv(volume="", faktor="", step="", jumlah="", status="", note="")
```

- Membuat `LOG_DIR` bila belum ada.

- Menulis header hanya jika file belum ada.
- Kolom: `timestamp`, `volume_ul`, `faktor_step_per_ul`, `step_per_capsule`, `jumlah_kapsul`, `status`, `keterangan`.
- Timestamp memakai format `YYYY-MM-DD HH:MM:SS`.
- Jika terjadi exception, pesan `[LOG ERROR] ...` dikirim ke serial monitor.
- Tidak ada mekanisme rotasi atau pembatasan ukuran file.

`detect_arduino_port()`

Mencari port pertama yang memenuhi salah satu kondisi berikut:

- Deskripsi mengandung `arduino`, `ch340`, `usb-serial`, `ft232`, atau `usb serial`.
- Fallback: deskripsi mengandung `usb` dan (`serial` atau `com`).

Fungsi ini tidak memverifikasi bahwa perangkat benar-benar Arduino atau firmware yang sesuai.

`safe_write(s)`

- Mengambil `serial_lock`.
- Menulis hanya jika `ser` ada dan `ser.is_open`.
- Menambahkan baris `TX → ...` ke monitor bila berhasil.
- Mengembalikan `True`/`False`; exception dicatat sebagai `[ERROR TX]`.

`send_init_command()`

Urutan kerjanya:

1. Membaca `entry_volume`, `jumlah_var`, dan `kal_var`.
2. Mengubah volume dan faktor menjadi float; jumlah menjadi integer.
3. Menampilkan warning bila volume/faktor tidak valid atau jumlah di luar 1–5.
4. Menolak volume > `MAX_VOLUME_UL`, menulis log `LOCK`, dan menampilkan error.
5. Menghitung `step_per_capsule = int(round(volume * faktor))`.
6. Menolak hasil step ≤ 0.
7. Menulis log `SENT`.
8. Membentuk perintah `<INIT:step;jumlah>`.
9. Menampilkan perhitungan pada monitor, memulai progress bar, dan mengeset status `"Sending INIT..."`.
10. Mengirim perintah melalui `safe_write()`.
11. Menjalankan thread `watch_timeout(6)` yang menghentikan progress setelah 6 detik atau saat status mengandung `DONE`/`ERROR`.
12. Bila pengiriman gagal, menampilkan error dan mengeset status `"Serial Disconnected"`.

Catatan: log `SENT` ditulis **sebelum** `safe_write()` dipanggil, sehingga log tetap menyatakan `SENT` meskipun pengiriman gagal.

`send_manual_command()`

- Membaca isi `entry_command`.
- Menolak perintah kosong.
- Menambahkan newline bila belum ada.
- Mengirim melalui `safe_write()`.

- Menghapus isi kolom hanya jika pengiriman berhasil.

Perintah manual tidak divalidasi dan dapat melewati batas `MAX_VOLUME_UL`, misalnya jika operator mengirim `<INIT:...>` secara langsung.

`handle_incoming_status(msg)`

Memetakan kata kunci berikut (tidak peka huruf besar/kecil):

Kata kunci	Status label	Warna
`READY`	`Status: READY`	lime
`BUSY`	`Status: BUSY`	orange
`FILLING` atau `FILL`	`Status: FILLING`	yellow
`DONE`, `FINISHED`, `COMPLETE`	`Status: DONE`	cyan
`ERROR` atau `FAIL`	`Status: ERROR`	red

Setelah pemetaan, kode selalu memanggil `log_to_csv(status="DONE", note="Dispensing complete")` (`HMI_FINAL.py:285`). Akibatnya setiap baris RX tercatat sebagai `DONE`, termasuk pesan READY, BUSY, atau error.

7. Alur Kerja (Logic Flow)

7.1 Startup

1. Modul diimpor dan konstanta/global diinisialisasi.
2. Widget Tkinter dibangun (`HMI_FINAL.py:350-464`), termasuk monitor, tombol, progress bar, dan status.
3. Tema gelap diterapkan melalui `apply_theme_dark()`.
4. Thread `serial_reader_thread` dijalankan sebagai daemon (`HMI_FINAL.py:469-470`).
5. `auto_reconnect_loop` dijadwalkan pertama kali setelah 500 ms (`HMI_FINAL.py:472`).
6. Saat berada di dalam `__main__`, folder `logs` dipastikan ada, teks `"HMI started."` ditambahkan ke monitor, lalu `root.mainloop()` dijalankan (`HMI_FINAL.py:489-496`).

```
Program start
  ├─ build UI
  ├─ apply_theme_dark()
  ├─ start serial_reader_thread (daemon)
  ├─ root.after(500, auto_reconnect_loop)
  └─ root.mainloop()
       ├─ auto_reconnect_loop setiap 2000 ms
       └─ event UI (tombol, input, close)
```

7.2 Auto-reconnect dan Refresh COM

Auto-reconnect (`HMI_FINAL.py:292-315`):

1. Jika `ser` kosong atau tidak terbuka, panggil `detect_arduino_port()`.
2. Bila port ditemukan, buka `serial.Serial(port, 9600, timeout=1)` di dalam lock.
3. Tunggu 1,5 detik agar perangkat melakukan reset.
4. Tampilkan pesan koneksi, set status `READY`, dan warna lime.
5. Bila gagal, catat error dan set `ser = None`.
6. Selalu menjadwalkan ulang dirinya setiap `RECONNECT_INTERVAL_MS` (2 detik) melalui `root.after`.

Refresh COM (`HMI_FINAL.py:430-457`) melakukan hal serupa secara manual, dengan jeda 1,2 detik, dan hanya jika belum terhubung. Jika sudah terhubung, monitor menampilkan `[Manual] Already connected.`

```
auto_reconnect_loop
  ├─ sudah terhubung? ── ya ──> jadwalkan ulang (2 dtk)
  └─ tidak
       ├─ detect_arduino_port()
       │    ├─ ditemukan → open serial → status READY
       │    └─ tidak     → tidak ada aksi
       └─ jadwalkan ulang (2 dtk)
```

7.3 Pengiriman perintah INIT

```
Klik "KIRIM KE ARDUINO"
  ├─ baca volume, faktor, jumlah
  ├─ validasi angka dan rentang
  │    └─ gagal → messagebox warning → stop
  ├─ volume > 10 µL?
  │    └─ ya → log LOCK + messagebox error → stop
  ├─ step = round(volume × faktor)
  ├─ step ≤ 0?
  │    └─ ya → messagebox error → stop
  ├─ log SENT
  ├─ cmd = <INIT:step;jumlah>
  ├─ progress.start() + status "Sending INIT..."
  ├─ safe_write(cmd + "\n")
  │    ├─ gagal → messagebox error + progress.stop()
  │    │           + status "Serial Disconnected"
  │    └─ berhasil
  │         └─ watch_timeout(6 dtk)
  │              ├─ status DONE/ERROR → progress.stop()
  │              └─ timeout          → progress.stop()
  └─ selesai
```

7.4 Penerimaan data serial

1. `serial_reader_thread` berjalan selama `running = True`.
2. Jika `ser` ada dan terbuka, thread mengambil `serial_lock` lalu memanggil `ser.readline()` dengan timeout 1 detik.
3. Data kosong → tidur `SERIAL_READ_SLEEP` (0,1 detik) lalu lanjut.

4. Data diterjemahkan ke teks (`errors="ignore"`), di-strip, lalu ditampilkan sebagai `RX ← ...`.
5. `handle_incoming_status()` dipanggil untuk memperbarui status.
6. Exception pembacaan dicatat sebagai `[RX ERROR]`, lalu thread tidur 0,5 detik.
7. Jika belum terhubung, thread tidur 0,2 detik.

```
serial_reader_thread
  └ loop selama running
      ├ ser terbuka?
      │    ├ tidak → sleep 0.2 s
      │    └ ya
      │         ├ readline() [lock, timeout 1 s]
      │         ├ data kosong → sleep 0.1 s
      │         └ data ada
      │              ├ decode + strip
      │              ├ append "RX ← ..."
      │              └ handle_incoming_status()
      │                   ├ update status_label/status_var
      │                   └ log DONE (selalu)
      └ ulang
```

7.5 Perintah manual

1. Operator mengetik perintah pada `entry_command`.
2. Tombol **Send Command** memanggil `send_manual_command()`.
3. Perintah kosong ditolak.
4. Newline ditambahkan bila belum ada.
5. Perintah dikirim ke serial dan kolom dibersihkan bila berhasil.

Contoh perintah manual yang dapat dikirim: `<M2;1600;1>`, `<HOME:ALL>`, `<HOME:3>`, `<READPOS>`.

7.6 Logging CSV

- Setiap pemanggilan `log_to_csv()` membuka file dengan mode append.
- Header ditulis hanya saat file belum ada.
- File disimpan di `logs/dispensing_log.csv` relatif terhadap direktori kerja.
- Log ditulis dari thread UI (INIT/LOCK) dan dari thread serial (status RX).
- Tidak ada lock khusus untuk penulisan log.

7.7 Tema

- Tema awal: gelap.
- `toggle_theme()` membalik `theme_dark`, memanggil `apply_theme_dark()` atau `apply_theme_light()`, lalu mengubah teks tombol menjadi `Light Mode`/`Dark Mode`.
- Tema mengubah warna root, style `TLabel`/`TButton`/`TFrame`, monitor, dan status bar.
- Beberapa widget ttk lain dapat tetap memakai warna tema `clam` karena tidak dikonfigurasi satu per satu.

7.8 Shutdown

1. Pengguna menekan tombol tutup jendela.
2. `on_closing()` menampilkan dialog konfirmasi.
3. Jika disetujui: `running = False`, `close_serial()`, lalu `root.destroy()`.
4. Thread reader dan watcher adalah daemon, sehingga berhenti bersama proses; tidak ada `join()` eksplisit.

8. Protokol Serial

8.1 Data yang dikirim HMI

Perintah	Format	Keterangan
INIT otomatis	`<INIT:stepPerCapsule;jumlah>`	Dikirim dengan newline di akhir.
Perintah manual	Teks bebas + newline	Tidak divalidasi HMI.

Contoh:

```
<INIT:20330;4>
```

Artinya 20.330 step per kapsul untuk 4 kapsul.

8.2 Data yang diharapkan HMI

HMI mencari kata kunci berikut pada setiap baris RX:

```
READY, BUSY, FILLING/FILL, DONE/FINISHED/COMPLETE, ERROR/FAIL
```

8.3 Perbandingan dengan firmware `v6_working`

Firmware `I_131_SingleFillSingleDispense_v6_working` tidak mengirim kata kunci bahasa Inggris tersebut. Output yang benar-benar dikirim antara lain:

Kondisi firmware	Output firmware	Cocok dengan HMI?
Startup	`Sistem siap. Masukkan perintah...`	Tidak.
Perintah diterima	`OK: INIT Accepted`	Tidak.
Selesai otomatis	`=== AUTO SELESAI ===`	Tidak (tidak mengandung `DONE`).
Format salah	`ERR: INIT Format Wrong`	Tidak (`ERR` bukan `ERROR`).
Nilai di luar rentang	`ERR: INIT Value Out of Range`	Tidak.
Progress internal	`=== PREPARE SELESAI...`, `=== PROSES KAPSUL n ===`	Tidak.

Konsekuensinya:

- `status_label` umumnya tetap `READY` setelah auto-reconnect.
- Progress bar tidak berhenti karena pesan DONE dari firmware, melainkan karena timeout 6 detik.
- Setiap baris RX tetap dicatat sebagai `DONE` akibat pemanggilan log tanpa syarat.

8.4 Kompatibilitas parser firmware

Firmware `v6_working`:

- Menandai perintah lengkap saat newline atau karakter `>` diterima.
- Mengambil isi di antara `<` dan `>`.
- Menerima `INIT:` dengan nilai step langsung (bukan µL).
- Memvalidasi jumlah kapsul 1–5 dan step > 0.

HMI mengirim newline setelah perintah, sehingga format tersebut kompatibel dengan parser firmware.

9. Format Log CSV

Lokasi: `logs/dispensing_log.csv`

Header:

```
timestamp,volume_ul,faktor_step_per_ul,step_per_capsule,jumlah_kapsul,status,keter
angan
```

Contoh isi:

```
timestamp,volume_ul,faktor_step_per_ul,step_per_capsule,jumlah_kapsul,status,keter
angan
2026-10-20 10:00:00,5.0,2033,10165,2,SENT,INIT sent to Arduino
2026-10-20 10:00:01,,,,,DONE,Dispensing complete
2026-10-20 10:00:02,12.0,2033,,2,LOCK,Volume melebihi batas maksimum
```

Status yang muncul pada kode:

Status	Kapan ditulis	Catatan
`SENT`	Sebelum pengiriman INIT	Tetap ditulis meskipun pengiriman gagal.
`LOCK`	Volume > 10 µL	Disertai keterangan batas maksimum.
`DONE`	Setiap baris RX	Ditulis tanpa syarat, termasuk untuk pesan non-DONE.
`ERROR`	Tidak terjangkau saat ini	Cabang error yang menulis log berada setelah cabang yang tidak dapat dicapai.

10. Cara Menjalankan

1. Pastikan Python 3 dan `pyserial` terpasang.
2. Unggah firmware Arduino yang kompatibel.
3. Tutup Serial Monitor Arduino IDE atau aplikasi lain yang memakai port yang sama.
4. Hubungkan Arduino melalui USB.
5. Jalankan:

```
py HMI_FINAL.py
```

6. Tunggu auto-reconnect (maksimal sekitar 2 detik) atau klik **Refresh COM**.
7. Pastikan monitor menampilkan `[Auto-Reconnect] Connected to COMx` dan status menjadi `READY`.
8. Isi volume, faktor kalibrasi, dan jumlah kapsul.
9. Klik **KIRIM KE ARDUINO**.
10. Pantau trafik pada Serial Monitor dan status pada label.
11. Periksa hasil pada `logs/dispensing_log.csv`.

11. Troubleshooting

Gejala	Kemungkinan penyebab	Tindakan
`[Manual] No suitable COM port found.`	Port belum terpasang, driver belum ada, atau kabel bermasalah.	Periksa kabel/driver, klik Refresh COM.
Terhubung ke port yang salah	Deteksi memakai heuristik dan memilih port pertama yang cocok.	Cabut perangkat USB-serial lain atau sesuaikan `detect_arduino_port()`.
Gagal membuka port	Port sedang dipakai Serial Monitor/aplikasi lain.	Tutup aplikasi yang memakai port tersebut.
Status tetap `READY` setelah INIT	Firmware tidak mengirim kata kunci yang dikenali HMI.	Gunakan pemetaan status yang sesuai atau sesuaikan firmware/HMI.
Progress berhenti sebelum proses selesai	Timeout watcher tetap 6 detik.	Tunggu proses firmware; jangan mengandalkan progress bar sebagai indikator selesai.
`Serial Error: Gagal mengirim ke Arduino.`	`ser` tidak ada/tidak terbuka.	Tunggu auto-reconnect atau klik Refresh COM.
Log tidak ditemukan	Aplikasi dijalankan dari direktori lain.	Jalankan dari direktori proyek atau cari `logs` di direktori kerja.